

システム管理(主題1.08)

アカウント管理

1. 这道题考查的是修改系统关键配置文件 `/etc/passwd` 的推荐方式。在 Linux 中，直接使用普通编辑器修改这类文件是非常危险的，因此有专门的工具来确保操作的安全性和正确性。

「/etc/passwd」の情報を変更するために最適なコマンドは **usermod** と **vipw**

`/etc/passwd`：存储用户基本信息（如 UID、GID、家目录、Shell 等）的关键文件。

文件锁定机制：为了防止多个进程同时修改该文件导致损坏，推荐使用具有锁定机制的工具。

专门命令：Linux 提供了如 `usermod` 等高层命令，以及 `vipw` 等安全编辑工具。

为什么选择 `usermod`？

- **安全性高**：它是一个高层命令，会自动处理语法检查和文件锁定。
- **功能明确**：专门用于修改现有的用户账户设置（如修改用户所属组、家目录等），这些改动会同步反映在 `/etc/passwd` 中。

为什么选择 `vipw`？

- **排他控制**：`vipw` 在编辑时会锁定 `/etc/passwd` 文件，防止其他管理员或进程同时修改。
- **语法检查**：在退出编辑并保存时，它会检查文件格式是否正确，如果格式有误会提醒你，防止系统因配置错误无法登录。

- **改用户，用 `usermod` 调参数；非要编辑，找 `vipw` 锁住门。**

2. 対話的ログインを禁止する方法として、最も適切なコマンドは **`usermod -s /bin/false hoge2`** と **`chsh -s /sbin/nologin hoge2`**

这道题考查的是如何通过修改用户的 Shell（壳程序）来禁止其进行 対話的ログイン（交互式登录）。

为什么这两个是正确的？

- **`usermod -s`**：`-s` (shell) 选项可以直接修改 `/etc/passwd` 中该用户的 Shell 字段。将其设为 `/bin/false` 后，用户尝试登录时会立即失败。
- **`chsh -s`**：`chsh` (change shell) 是专门用于修改 Shell 的命令。将其设为 `/sbin/nologin` 是 Linux 系统中禁用服务的标准做法。

错误选项解析

- **`userdel hoge2`**：这直接删除了用户。题目要求的是“禁止登录”，通常暗示保留账户（如跑服务用）但不允许人去登录。

- `passwd -nologin hoge2` : `passwd` 命令没有 `-nologin` 选项。
- `usermod -L hoge2` : `-L` (Lock) 是**锁定密码**。虽然这会让用户因为密码无效而无法登录，但如果该用户配置了**SSH 密钥登录**，他依然可以登录成功。因此，修改 Shell 才是最彻底、最适合“禁止交互式登录”的方法。
- **禁止登录改 Shell, nologin 和 false。chsh 直接换，usermod 加个 s。**

3. ユーザ・グループアカウントに関するコマンド

コマンド	説明
passwd	ユーザのパスワードを変更
chage	パスワードの有効期限を変更
useradd	ユーザアカウントを新規作成
userdel	ユーザアカウントを削除
usermod	既存のユーザアカウントの設定を変更
groupadd	グループアカウントを新規作成
groupdel	グループアカウントを削除
groupmod	既存のグループアカウントの設定を変更
id	ユーザのUIDやGIDを表示
groups	ユーザの所属するグループを表示
gpasswd	グループのパスワードやメンバーを設定

核心动作口诀（三字经版）

- 增用 add，删用 del，修改 mod。
- 前面加 user，后面管账户。
- 前面加 group，后面管群组。

细节功能口诀（谐音关联版）

- Passwd 改密码，Chage 期限有效期。
- ID 看 ID，Groups 看多组。
- Gpasswd (G前缀) 组管家。

既存のグループアカウントの設定を変更するコマンドは「groupmod」です。

groupmodコマンドの書式と主なオプションは以下の通りです。

groupmod [オプション] グループ名

オプション	説明
-g GID	GID(グループID)を変更
-n グループ名	グループ名を変更

Groupmod (群组翻修机), 改号改名靠两招: -g (GID) 换编号, -n (Name) 改名称。

 例题: 「/etc/skel/」以下のファイルが、雛形としてホームディレクトリにコピーされる場合のあるコマンドは**useradd**

当你执行 `useradd -m 新用户名` 时, 系统会执行以下流水线操作:

1. 在 `/etc/passwd` 等文件中登记新用户信息。
2. 在 `/home` 下创建一个新文件夹。
3. **关键点:** 从 `/etc/skel/` 拷贝所有预设文件到这个新文件夹, 并把所有权交给新用户。这样新用户第一次登录时, 就已经有了标准的 shell 环境。

新开账户 `useradd`, 拷贝模板找 `skel`

为什么其他选项无关?

- **groupadd:** 只负责创建“组”。组是没有家目录的, 所以不需要拷贝模板文件。
- **usermod:** 用于修改**已经存在**的用户。因为用户已经有了家目录, 所以不会再次触发从 `/etc/skel` 拷贝文件的动作。

5. 题目: ユーザuser1を削除する際、ホームディレクトリも一緒に削除したい。適切なコマンドは次のうちどれか。この際、アカウントが使用中であっても削除できるものを選び。

这道题考查的是 `userdel` 命令在删除用户时的**高级组合选项**。

关于你的疑问: “为什么不能用 `-d`? `-d` 不是代表 Directory 吗?” **其实这里有一个容易弄混的逻辑陷阱:** 在 `useradd` 和 `usermod` 中, `-d` 确实代表 **directory** (指定家目录)。但是在 `userdel` (删除命令) 中, 官方设计的“连同家目录一起删除 (**remove**)”的选项是 `-r`, 而 `userdel` 甚至根本没有 `-d` 这个选项!

知识点总结

- **userdel:** 用于删除用户账号。
- **-r 选项 (remove):** 连同用户的ホームディレクトリ (家目录) 和邮件池一起删除。
- **-f 选项 (force):** 强制 (强制) 执行。即使该用户当前正处于登录状态 (使用中), 或者该家目录已经被其他用户占用, 也会强行进行删除。

为什么必须是 `-rf` ?

题目中有两个核心要求:

- a. 「ホームディレクトリも一緒に削除」 → 需要使用 `-r` 。
- b. 「アカウントが使用中であっても削除できる」 → 需要使用 `-f` (Force) 。
- c. 将这两个选项组合起来, 就是 `-rf` (或者 `-r -f`) 。

 **错误选项解析**

- `userdel -df user1`：错误。如前所述，`userdel` 没有 `-d` 选项，系统会报错 `invalid option -- 'd'`。
- `userdel -r user1`：虽然能删家目录，但如果用户正处于登录“使用中”，命令会失败并报错“`user user1 is currently used by process...`”，不满足题目第二个条件。
- `userdel -ar user1`：`userdel` 没有 `-a` 选项。
- `usermod -u / -d`：`usermod` 是用来修改用户的，不是用来删除用户的。

考试小秘籍

a. 字母功能错位陷阱：这是 Linux 考试最喜欢挖的坑。

- 建楼/改楼（`useradd` / `usermod`）时：找 `-d`（指定 **d**irectory 地址）。
- 拆楼（`userdel`）时：找 `-r`（**r**emove 掉目录）。

b. 慎用 `-f`：在实际生产环境中，`userdel -rf` 非常危险，因为强行删除正在运行的用户可能会导致该用户运行的后台进程产生大量死锁或孤儿进程。但在考试中，看到“使用中（使用中）+ 强行”，闭眼选 `-f`。

6. 这道题考查的是对 Linux 系统中存储加密密码的极其敏感的文件——`/etc/shadow` 安全性的理解。

根据 Linux 的安全实践，针对 `/etc/shadow` 的权限设定通常遵循以下原则：

- a. **000** (`-----`)：这是许多现代发行版（如 Ubuntu, CentOS）的默认设置。即使是 root 也没有显式的 r/w 权限，但在 Linux 中 root 拥有绕过文件权限检查的特权（Capabilities），因此依然可以读写。这能最大程度防止误操作。
- b. **400** (`-r-----`)：这也是一种常见的设置，意味着只有 root 用户可以读取密码散列，其他人完全无法接触。
- c. **600** (`-rw-----`)：允许 root 用户读取和修改密码，而普通用户依然没有任何权限。

想象一下 `/etc/shadow` 是家里的**保险箱密码本**：

- **000/400/600**：就像保险箱锁得死死的，只有**户主 (root)** 才能打开。别人连看一眼的机会都没有。
- **444/666/777**：就像把密码本直接贴在大门外。哪怕你加密了，小偷也可以抄走慢慢研究。

口诀：影子文件 shadow，全家安全它把口。末尾数字必为零，不给闲人留窗口。

考试小秘籍

d. **观察末位**：对于 `/etc/shadow` 或 `/etc/shadow-` 这种敏感文件，其权限的后两位（Group 和 Others）**必须是 0**。任何后两位不是 0 的选项直接排除。

e. **root 的特权**：记住 `000` 权限对 root 并不起完全阻拦作用，这在考试中是一个常见的“陷阱点”。

- f. 配套考点：通常会对比 `/etc/passwd`（权限通常为 `644`，全员可读）和 `/etc/shadow`（权限严格，仅 root 可读）

7. useraddコマンド

ユーザアカウントを作成するには、「useradd」コマンドを使用します。

useraddコマンドの書式と主なオプションは以下の通りです。

useradd [オプション] ユーザ名

オプション	説明
-g グループ名 GID	プライマリグループを指定
-G グループ名 GID	プライマリグループ以外に参加する補助グループ (suppelementary groups)を指定
-d ディレクトリ	ホームディレクトリを指定
-m	ホームディレクトリを作成(存在しない場合)
-c コメント	コメントを指定
-s シェル	ログインシェルを指定
-D	アカウントのデフォルト値を表示または設定

小 g (group) 主組，大 G (Groups) 輔，

小 d (directory) 路径，小 m (make) 盖屋。

小 c (comment) 备注，小 s (shell) 登入，

大 D (Default) 默认，规则全记住！

8. グループに関する情報は「/etc/group」ファイルに格納されています。

グループ名:グループパスワード:GID:グループのメンバーリスト（核心结构：`组名:组密码占位符:GID:辅助组用户列表`。）

- ・グループ名：一意のグループ名
- ・グループパスワード：グループのパスワード
- ・GID：グループのGID
- ・グループのメンバーリスト：グループに所属するメンバー。複数の場合は「,」で区切る



注意：グループのメンバーリストには所属メンバーが批載されていない場合がある（组的成员列表中可能没有记载所属成员。）

为什么正确答案说“可以没有成员”？

在 Linux 中，末尾的成员列表只记录把该组当做 辅助组 的用户。

- 场景 A：当你用 `useradd smith` 新建用户时，系统会默认创建一个同名的主组 `smith`。此时查看 `/etc/group`，你会发现 `smith:x:1001:` 后面是空的。因为对 `smith` 来说这是他的主组，不需要在末尾重复写他的名字。

- **场景 B**：新创建了一个空组（如 `groupadd testgroup`），里面还没加任何人，末尾自然也是空的。因此，列表中没有成员是非常正常且普遍的情况。

ユーザのアカウント情報は「/etc/passwd」ファイルに格納されています。

ユーザ名：パスワード：UID：GID：コメント：ホームディレクトリ：ログインシェル

- ユーザ名：一意のユーザ名
- パスワード：ユーザのパスワード。
- UID：ユーザのUID（ユーザID）。UIDはユーザを識別するための一意のID
- GID：ユーザのプライマリグループのGID（グループID）。
GIDはグループを識別するための一意のID
- コメント：ユーザに関する任意のコメント（省略可）
- ホームディレクトリ：ユーザのホームディレクトリ
- ログインシェル：ユーザのログインシェル（ログイン時に使用されるデフォルトのシェル）

9. ユーザのUID（ユーザID）や、所属するグループのGID（グループID）を確認するには「id」コマンドを使用します。

id [オプション] [ユーザ名]

オプション	説明
-u	UID(ユーザID)を表示
-g	ユーザのプライマリグループのGID(グループID)を表示
-G	所属する全てのグループのGIDを表示

id 命令查身份，选项全在首字母：-u (User) 只看我（UID），-g (group) 瞧主组（GID），-G (Groups) 扫全组（所有 GID）。

ユーザ名を省略すると、コマンドを実行したユーザの情報が表示されます。

また、オプションを省略すると、UIDと所属する全てのグループのGIDを表示します。

```
#id ken
uid=1104(ken) gid=1104(ken) groups=1104(ken), 3(adm), 982(rfkill)
```

プライマリ プライマリ 補助 補助

所属する全てのグループ

6. アカウントとパスワードに関する情報が「/etc/passwd」と「/etc/shadow」に分けて管理されている理由は他ユーザのパスワードの閲覧や解読を防ぐ

这道题考查的是 Linux 系统安全设计中的核心逻辑：`/etc/passwd` 与 `/etc/shadow` 分离的根本原因。

在 Linux 早期版本中，加密后的密码也存放在 `/etc/passwd` 中。但由于系统运行需要，该文件必须对所有用户开放**读取权限**（否则无法通过 UID 获取用户名）。为了解决这个安全隐患，设计者引入了 **Shadow Password** 机制。

- `/etc/passwd`：保存 UID、GID、Shell 等基础信息。权限通常为 **644**（全员可读），方便程序查询用户信息。
- `/etc/shadow`：保存加密后的密码（Hash）。权限极其严格（通常为 **000/400/600**），仅 root 可读，从而实现了信息的物理隔离。

物理隔离：如果密码存放在 `644` 权限的 `/etc/passwd` 中，任何人都可以将其拷贝走。虽然是加密的，但攻击者可以利用离线暴力破解或“彩虹表”在本地尝试解密。

防范解密：将密码转移到普通用户“完全看不见”的 `/etc/shadow` 中，从根源上切断了普通用户获取 Hash 值的途径，极大地提高了安全性。

如果卡号和密码印在一起，大家就都能拿去破解了。所以要“号、密分离”。

权限关联：记住 `/etc/passwd` 末尾权限是 **4 (Read)**，而 `/etc/shadow` 末尾权限是 **0 (None)**。

7. 填入后为 `getent passwd`。这条命令的作用是列出系统中所有的用户信息，其输出格式与 `/etc/passwd` 文件完全一致。

功能定位：它是专门设计用来“取得（获取）”各类系统数据库信息的标准工具。

命令	对应查询的文件	作用
<code>getent passwd</code>	<code>/etc/passwd</code>	获取所有用户信息
<code>getent group</code>	<code>/etc/group</code>	获取所有组信息
<code>getent hosts</code>	<code>/etc/hosts</code>	获取 IP 与主机名映射
<code>getent passwd root</code>	<code>/etc/passwd</code>	仅获取 root 用户的详细信息

要拿信息用 `getent`，后面跟着数据库。`passwd`、`group` 都能查，不用 `cat` 去翻书。

 考试小秘籍

- a. **关键词锁定**：看到题目要求从某个“数据库名称”（如 `passwd`，`group`）获取信息，优先考虑 `getent`。
- b. **区分 id 命令**：`id` 主要是针对“当前用户”或“指定用户”的，而 `getent passwd` 倾向于查看“整个账本”。
- c. **实用性**：在排查“为什么某个用户明明存在却无法登录”时，`getent passwd 用户名` 是比查看文件更专业的方法。

8. 题目：「/etc/nsswitch.conf」の設定内容を確認するのに役立つコマンドは**getent**

这道题考查的是 `/etc/nsswitch.conf`（Name Service Switch，名称服务切换配置）与系统信息检索工具之间的联动关系。

💡 知识点总结

- `/etc/nsswitch.conf` 的作用：它就像是 Linux 系统的“寻宝指南”。当系统需要查找用户信息（`passwd`）、组信息（`group`）或主机名（`hosts`）时，这个文件决定了系统先去哪里找、后去哪里找（例如：先看本地的 `/etc/passwd` 文件，还是先问网络上的 `LDAP` / `NIS` 服务器）。
- `getent (get entries)`：我们在前面的卡片学过，它是专门用来从系统数据库提取条目的工具。
- 两者的联动：`getent` 命令在执行时，完全遵循 `/etc/nsswitch.conf` 里规定的顺序和源去查找数据。因此，通过比较 `getent` 的输出结果，最能直接验证 `/etc/nsswitch.conf` 的配置是否生效。

为什么选择 `getent` ？

- 验证配置：假设你在 `/etc/nsswitch.conf` 里把 `passwd` 的查找顺序改成了先查网络 `LDAP` 再查本地文件。当你输入 `getent passwd` 时，它就会乖乖按照这个新顺序去获取所有用户。如果配置错了，`getent` 就捞不到数据。所以它是公认检验该配置文件最有效的命令。
9. 「/etc/shadow」ファイルには、暗号化されているパスワード（シャドウパスワード）や、パスワードの有効期限に関する情報が格納されています。

以下は「/etc/shadow」ファイルの書式と各項目の説明です。

下記の「日数1」で最終更新日が分かります。

ユーザ名：パスワード：日数1：日数2：日数3：日数4：日数5：日数6：フラグ

- ユーザ名：一意のユーザ名（`/etc/passwd`と同様）
- パスワード：暗号化されたパスワード
- 日数1：1970/1/1から、パスワードが最後に更新された日までの日数
- 日数2：パスワードが変更できるようになるまでの日数
- 日数3：パスワードを変更しなければならなくなる日までの日数
- 日数4：パスワードの有効期限が切れる前に警告をだす日数
- 日数5：パスワードの有効期限切れからアカウントが使用できなくなるまでの日数
- 日数6：1970/1/1から、アカウントが使用できなくなる日までの日数
- フラグ：未使用

10. 这道题考查的是如何通过修改用户的**辅助グループ（Supplementary Groups / 补助组）**，来将用户从某个特定组中移除。这属于多组管理中非常经典的一个命令操作。 \

题目：あるユーザが3つのグループ（A,B,C）に属しているが、Bグループからそのユーザを削除したい。正しいのはどれか。ただし**Bグループはユーザのプライマリグループではないもの**とする。

为什么选择 `usermod -G` ?

- **命令逻辑**：根据我们之前学过的口诀，大写 `-G` 代表辅助组（Groups复数）。
- **操作方式**：因为题目明确了 B 不是プライマリグループ（主组），所以它一定属于辅助组。要调整现有的辅助组状态，必须使用修改用户的命令 `usermod -G`。

✗ 错误选项解析

- **groupmod -g / groupmod -G**： `groupmod` 是用来修改**组本身**的属性（如改组名 `-n` 或改GID `-g`），它没有 `-G` 选项，也无法直接用来管理组内的成员。
- **usermod -g**：小写 `-g` 是用来修改プライマリグループ（主组）的。题目已经说了 B 不是主组，所以不选它。
- **userdel -G**： `userdel` 是用来**删除用户账号**的，根本没有 `-G` 选项。

口诀：调动用户 `usermod`，辅助群组大 G 控。覆盖写全留下的，不写名字自动走。

✍ 考试小秘籍

- 覆盖陷阱**：在实战或考试中，执行 `usermod -G` 时一定要小心。如果你只想把用户加入一个新的组 D，但漏写了原本属于的 A 和 C（比如只写了 `usermod -G D 张三`），那么张三原本的 A 和 C 组身份会**全部丢失**！
- 安全添加的备选（延伸）**：为了防止上面的误操作，很多现代 Linux 系统允许使用 `-aG`（Append Groups）来做纯添加，这样就不会覆盖旧组了。但如果要“删除”，依然需要用 `-G` 重新覆盖定义。
- gpasswd 的快捷方式**：除了这道题的官方解法外，我们在前面的卡片中学过的 `gpasswd -d 用户名 组名` 也可以直接把用户从组里删掉（d 代表 delete），这也是非常高频的替代考点哦！

既存のユーザのアカウント情報を変更するにはusermodコマンドを使用します。

`usermod [オプション] ユーザ名`

オプション	説明
-d ディレクトリ	ホームディレクトリを指定
-g グループ名 GID	プライマリグループを指定
-G グループ名 GID	プライマリグループ以外に参加する補助グループ(supplementary groups)を指定
-aG グループ名 GID	現在の補助グループ設定を維持しつつ追加
-s シェル	ログインシェルを指定
-L	パスワードをロックする
-U	パスワードをアンロック(解除)する
-p 暗号化済みのパスワード	暗号化済みのパスワードを設定する

根据图片 `image_70ab68.png`，`usermod` 用于修改现有账户，核心选项如下：

- **-d**：修改 **ホームディレクトリ**（家目录路径）。
- **-g / -G**：修改 **プライマリグループ**（主组）与 **補助グループ**（辅助组）。
- **-aG**：追加 (Append) 辅助组，维持现有的辅助组设定不被覆盖。
- **-s**：修改 **ログインシェル**（登录 Shell）。
- **-L / -U**：**ロック (Lock / 锁定)** 与 **アンロック (Unlock / 解锁)** 密码。
- **-p**：直接设定 **暗号化済み（加密后）** 的密码。

核心口诀（英文关联版）

小 d (directory) 搬家换地址，小 g 主组 大 G 辅。

想要加组不覆盖，必须小 a 连大 G (-aG)。

小 s (shell) 换个新钥匙 (Shell)，大 L (Lock) 锁死，大 U (Unlock) 开。

小 p (password) 强塞加密密。



例题：ユーザのログインシェルを変更したい。適切なコマンドは **usermod -s**

11. グループに関する情報は「/etc/group」ファイルに格納されています
12. 这道题确实非常经典且综合，它考查的是 Linux 系统中“暂时停用账户（アカウントの凍結 / 锁定）”的多种实现方法。

题目说的是“年末年始の休暇の間（元旦假期期间）”，这意味着假期结束后还要恢复使用，所以**绝对不能使用** `userdel` 删除账号。在 Linux 中，这种“短暂停用”在技术上被称为「**アカウントのロック（账户锁定）**」。

要实现“短暂停用（锁定）”一个账户，核心的思路有两种：

- a. **使用命令自动锁定**：通过专用命令在底层修改密码字符串。

- b. **手动修改配置文件**：直接编辑存储密码的文件，在加密密码的最前面加上一个特殊符号（如 `!` 或 `*`），使系统在校验密码时直接判定为“无效密码”。

为什么命令选项是对的？

- `passwd -l user3`：这里的 `-l` 代表 **Lock**（小写）。它的作用是在 `/etc/shadow` 文件的密码字段前加一个 `!`，从而锁定用户。
- `usermod -L user3`：这里的 `-L` 代表 **Lock**（大写）。它与 `passwd -l` 的效果完全一样，也是我们在上一张卡片学过的“安保大挂锁”。

为什么文件修改选项也是对的？

- 「`/etc/shadow`」的修改：现代 Linux 密码存在这里。如果在密文第1个字符前加 `!` 或 `*`，系统认证时就会认为这个密码格式失效，从而拒绝登录。
- 「`/etc/passwd`」的修改：虽然现代系统不用它存密码，但如果有些老系统（或者没有启用 Shadow 的系统）把密码存在 `passwd` 中，在它的密码字段（第2列）加上 `!` 或 `*` 也能起到同样的锁定效果。因此在理论考试中，这也是一个正确方法。

- **：短暂停用叫 Lock，大小 L 都能锁。手动修改影/密文，首字加个叹星号。**

13. 「`/etc/passwd`」は、ユーザのアカウント情報が格納されているファイルです。パーミッションは「644 (-rw-r--r--)」で一般ユーザによる読み出し権限が付与されています。

14. 这道题考查的是通过输出结果的**字段结构特征**来辨识 Linux 的核心配置文件。

```
user1:$1$0nrwy01S$3nT5EPyTjF/0JQEbIp.xn/:14552:0:99999:7:::
user2:$1$m1BRk/Fw$YFPj9gxUmwjSTQQDh5.Au1:14552:0:99999:7:::
user3:$1$a//P2VIX$6ZKNIztcFDbUkVk6PIpmi1:14552:0:99999:7:::
user4:$1$8nd/V436$StxVwE4QlqVWPDks7maIF0:14552:0:99999:7:::
user5:$1$8hMINIVS$gtjcMOVouDMCnIkQ8IHk7.:14552:0:99999:7:::
user6:$1$Zpwvu0TC$ZXe9zDSbGy5zkx zrEVMIB1:14552:0:99999:7:::
user7:!!:14552:0:99999:7:::
```

💡 知识点总结

- `/etc/shadow`（シャドウパスワードファイル）：每一行代表一个用户，字段之间用 `:`（冒号）分隔，总共有 9 个字段。
- 核心特征：
 - a. 第二个字段是一长串包含 `$1$` 或 `$6$` 开头的乱码（这是**加密后的密码哈希值**）。
 - b. 如果账户被锁定了，密码字段会显示为 `!` 或 `!!`（如图片中的 `user7`）。
 - c. 后面跟着一串数字（如 `14552:0:99999:7`），分别代表密码修改日期、最小/最大有效天数和警告天数。

15. 下記の結果を得るコマンドは `cat /etc/passwd`

```
user1:x:1001:100::/home/user1:/bin/bash
user2:x:1002:100::/home/user2:/bin/bash
user3:x:1003:100::/home/user3:/bin/bash
```

`/etc/passwd`（ユーザー設定ファイル）：每一行代表一个用户账户，字段之间用 `:`（冒号）分隔，总共有 7 个字段。

核心特征（长相）：

- a. 第二个字段统一固定为一个 `x`（代表密码被影子文件 `/etc/shadow` 接管了）。
- b. 后面紧跟着两个数字，分别是 **UID（用户ID）** 和 **GID（主组ID）**。
- c. 最后两个字段通常是明显的 `/home/用户名`（家目录路径）和 `/bin/bash`（登录 Shell）。

👍 例题：「`/etc/passwd`」ファイルの内容と、「`/etc/shadow`」ファイルの内容について、「`/etc/passwd`」ファイルのパスワード欄に「`x`」とある場合、そのユーザのパスワードは「`/etc/shadow`」に暗号化されて格納されている

16. 対話的ログインを禁止する方法として適切なものはログインシェルに`/sbin/nologin`を設定する
和 ログインシェルに`/bin/false`を設定する

交互式登录的本质：当用户成功输入密码登录系统后，系统必须启动一个 **Shell**（如 `/bin/bash`）来让用户能够输入命令、进行交互。

禁止交互式登录的核心：直接把他的登录 Shell 改成两个特殊的“假 Shell”：

- `/sbin/nologin`：礼貌拒绝。拒绝登录并优雅地提示一行字：“*This account is currently not available.*”（该账户目前不可用）。
- `/bin/false`：无情拒绝。什么提示都不给，登录直接返回失败并退出。

在创建或修改用户时，通过 `useradd -s` 或 `usermod -s` 把用户的 Shell 路径指向这两个文件，就能彻底切断该账户的交互式登录可能。

17. `useradd` コマンドでオプションを指定せずにユーザを作成する際、使用されるデフォルトの値が格納されているファイルは`/etc/default/useradd`

`/etc/default/useradd`：该文件存放着 `useradd` 命令的缺省值（出厂设置）。

其实，在前面卡片中我们学过一个查看默认值的选项：`useradd -D`。

当你执行 `useradd -D` 时，系统在后台干的事情就是直接去读取并把 `/etc/default/useradd` 的内容打印在屏幕上。

通过 `cat /etc/default/useradd` 或 `useradd -D`，你通常会看到类似下面的配置：

口诀：不加参数 `useradd`，默认配置哪里找？`etc` 下面设个 `default`，`useradd` 文件里边跑。

配置文件中的变量	含义说明	默认示例
HOME=	默认家目录建在哪个大目录下	/home
SHELL=	默认给用户分配什么钥匙 (Shell)	/bin/bash
SKEL=	默认从哪里拷贝“样板间”配置文件	/etc/skel
INACTIVE=	密码过期后多少天冻结账号	-1 (代表永不冻结)

另一个联动文件（延伸常考）：除了 `/etc/default/useradd` 之外，还有一个定义用户创建时 UID/GID 范围、密码加密算法的文件叫 `/etc/login.defs`。

- 区分方法：`useradd -D` 只能查看和修改 `/etc/default/useradd` 的内容；而 `/etc/login.defs` 是更底层的、管整个登录系统（包含 `login`, `passwd` 等多个命令）的全局定义。

18. Linuxにおける **グループの削除 (groupdel)** 命令を実行した際の、安全のための制限（バリデーション）に関する挙動を考査するものです。

groupdel コマンド：既存のグループアカウントを削除するコマンドです。

为什么执行后会失败？

当 root 用户尝试执行 `# groupdel sales` 时，系统会立刻去检查 `/etc/passwd` 文件。一旦发现有用户（如 smith）的主组依然是 `sales`，系统会拒绝执行并直接报错：`groupdel: cannot remove the primary group of user 'smith'` 因此，该命令不会对系统做任何修改。

高频考点：这是 LPIC-1 考试中关于账号管理的必考逻辑题。只要题目中出现“某个组是某人的**プライマリグループ**”，然后再问你**能不能直接 groupdel**，答案一律是“拒绝/无法删除”。

如何正确删除：如果考试问你“如何才能顺利删除 `sales` 组”，正确步骤是：

- 先执行 `usermod -g 新组名 smith`（把 smith 挂到别的承重墙上）。
- 再执行 `groupdel sales`（此时墙空了，可以安全拆除）。

19. 这道题考查的是我们在前面口诀里反复提到过的、多组管理中**最核心、最容易翻车**的黄金组合选项：`usermod -aG`。

在 Linux 中，`-a` 代表 **append (追加)**。它的唯一使命就是作为“安全保护锁”，阻止大写 `-G` 的彻底覆盖行为。

为什么必须选 `usermod -aG`？

题目有两个硬性指标：

- 「登録済のグループを維持しつつ」（保持已注册的组不变）→ 需要 `-a` (append)。
- 「追加登録したい」（追加注册辅助组）→ 需要 `-G` (Groups复数)。

c. 合起来就是 `-aG`（通常写在一起，也可以写成 `-a -G`）

20. ユーザのパスワードを設定・変更するコマンドは `passwd` と `usermod -p`

💡 知识点总结

- `passwd`：设置与修改密码的**官方正牌工具**。它的特点是“体贴”。你只需要输入明文（比如 `123456`），它会自动帮你加密并存入系统。
- `usermod -p`：修改用户属性的**全能工具箱**（`-p` 代表 password）。它的特点是“高冷、死板”。它不具备加密功能，你必须直接喂给它已经加密好的乱码（密文），直接写明文会导致用户无法登录。

🧠 密码设置命令记忆口诀

密码正牌 `passwd`，输入明文自动转。

全能修改 `usermod`，带上小 `p` 塞密文。

一个体贴自动加，一个高冷必须加密码！

21. 这道题考查的是 `useradd` 命令在不同 Linux 发行版（特别是 **Debian 系列**，如 Ubuntu、Debian）下的行为差异以及创建家目录的选项。

在 Red Hat 系列（如 CentOS, RHEL）中，执行 `useradd` 默认就会自动创建家目录。但在 **Debian 系列（如 Ubuntu）** 中，默认行为是**不创建家目录**的。

- `-m` 选项（**make home**）：强制要求系统在创建用户时，必须同时创建（作成）用户的ホームディレクトリ（家目录）。
- **样板间复制机制**：只要触发了 `-m` 创建家目录，系统就会自动将 `/etc/skel/` 目录下的所有隐藏配置文件（如 `.bashrc`，`.profile` 等样板文件）拷贝到新用户的家目录下。

ジョブスケジューリング

1. 「/etc/cron.allow」と「/etc/cron.deny」ファイルが存在しない場合、`crontab`コマンドを使用できるユーザは**rootユーザ**

这道题考查的是 Linux 系统中 `crontab` 命令的**访问控制权限（利用制限）** 的默认判定逻辑。

当控制权限的两个核心文件都不存在时，系统的保底策略会变得非常严格。

1. 如果有 <code>cron.allow</code>	👉	只有名单里的人可以用。（不管有没有 <code>deny</code> 都不用看了）
2. 如果没有 <code>allow</code> ，但有 <code>cron.deny</code>	👉	名单外的人可以用。（名单里的人被禁止）
3. 如果两个文件都不存在（本题）	👉	🔒 只有 root 用户 可以用。（普通用户全员禁止）

2. 这道题考查的是 `anacron` 的配置文件 `/etc/anacrontab` 中，用于防止大量定时任务在同一时间爆发而设定的**全局随机延迟变量**。

anacron 的作用：与二十四小时开机的 `cron` 不同，`anacron` 专为“经常关机的电脑或笔记本”设计。它在开机后会去检查过去有哪些定时任务因为关机漏掉了，并自动补回来。

为什么要随机延迟：如果电脑一开机，所有漏掉的任务同时排山倒海般地涌上来，CPU 可能会瞬间飙到 100%。为了错开峰值，系统引入了全局随机延迟变量。

RANDOM_DELAY：这就是控制全局最大随机延迟时间的变量名。

单位规范：它的单位固定是 分（Minute），并且在赋值时只能写纯数字，绝对不能带 `min` 这种单位后缀。

填入 `RANDOM_DELAY=30` 后，意味着系统在执行任何一个 `anacron` 任务之前，都会在 0 到 30 分钟之间随机抽取一个等待时间，从而完美错开所有任务的起跑线。

数字陷阱（重点）：很多同学容易把这个选项和 `cron` 的秒级延迟搞混。记住 `anacron` 的最小时间管理颗粒度是“天”，所以它的变量单位全都是“分钟”。看到 1800（秒）或者带 `min` 的直接划掉。

变量名		作用与单位	考试注意点
<code>RANDOM_DELAY=</code>		最大随机等待时间（单位：分）	必须是纯数字。
<code>START_HOURS_RANGE=</code>		允许执行任务的时间段（如 3-22）	规定只能在每天的几点到几点之间补任务。

「/etc/anacrontab」において、ジョブが実行される時間帯を制限するためのパラメータは `START_HOURS_RANGE`

3. 这道题考查的是 Linux 系统中 `cron` 定时任务系统默认的周期性执行目录（定期実行ディレクトリ）。

cron種別	設定ファイル・ディレクトリ	内容	編集方法	ジョブの実行権限
ユーザ用	/var/spool/cron/ユーザ名 ※ディストリビューションによっては /var/spool/cron/crontabs/ユーザ名	ユーザ用の設定ファイル	crontabコマンド	crontabコマンドを使用した ユーザの権限で実行
システム用	/etc/crontab	システム用の設定ファイル 主に以下の4つのディレクトリに 置かれたスクリプトを実行	viなどのテキスト エディタ	設定ファイルで指定した ユーザの権限で実行 *主にrootユーザ
	/etc/cron.hourly/	毎時実行するスクリプトを置く		
	/etc/cron.daily/	毎日実行するスクリプトを置く		
	/etc/cron.weekly/	毎週実行するスクリプトを置く		
	/etc/cron.monthly/	毎月実行するスクリプトを置く		
	/etc/cron.d/	サービス個別のジョブ実行を 定義した設定ファイルを置く		

系统已经帮我们按时间周期划分好了不同的“投递箱”，我们只需要根据名字就能一眼看穿！
在 `/etc/crontab`（系统级定时任务配置文件）中，默认配置了专门的机制（通常利用 `run-parts` 命令），会自动去定期执行以下五个目录下的所有脚本：

- `/etc/cron.hourly/`：每小时（毎時）执行一次。
- `/etc/cron.daily/`：每天（毎日）执行一次。
- `/etc/cron.weekly/`：每周（毎週）执行一次。

- `/etc/cron.monthly/`：每月（每月）执行一次。
- `/etc/cron.d/`：用来存放任何**自定义周期**的定时任务配置文件。

定时任务不用编，周期目录名字显。每小时执行找 hourly，天、周、月、d 各不相同！

✗ 错误选项解析

- `/etc/cron.d/`：它不是按照固定时间（如每小时、每天）来自动运行里面脚本的。它是用来放**标准的 crontab 格式配置文件**的。比如你想设置一个“每隔 3 分钟运行一次”的奇葩任务，就得写个配置文件扔到这个目录里。

绝对路径完整性：LPI 考试中，这几个目录的路径必须牢记，全部都在 `/etc/` 目录下。如果考填空题，千万别漏掉前面的 `/etc/` 和末尾的 `/`

执行权限（高频连带考点）：如果你把一个写好的 Shell 脚本放进了 `/etc/cron.hourly/`，但发现它到了整点**根本没有执行**，99% 的原因是因为你忘了用 `chmod +x 脚本名` 给它赋予**可执行权限**！系统里的 `run-parts` 命令会自动无视没有可执行权限的文件。



题目：ユーザのcrontabファイルが置かれるディレクトリは/var/spool/cron/

在 Linux 系统中，普通用户通过 `crontab -e` 编写的定时任务，并不会直接写进 `/etc/crontab`。为了防止互相干扰和越权，系统会为每个用户生成一个独立的**用户级 crontab 文件**。

- 该文件的保存目录是：`/var/spool/cron/`（部分 Debian/Ubuntu 系统为 `/var/spool/cron/crontabs/`）。
- **文件名**：直接就是用户的**用户名**（例如 `user1` 的定时任务就存在 `/var/spool/cron/user1` 里）。

娃儿 → 死鱼 → 缸 → `/var/spool/cron`

个人任务 crontab，不跟系统挤一处。

娃儿死鱼扔进缸（var/spool/cron），用户名字当外壳！

📝 考试小秘籍

1. **填空题大坑**：考试如果考填空题，一定要注意开头和末尾的斜杠（`/`）。标准大纲里标准的目录路径写为 `/var/spool/cron/`。
2. **权限安全**：因为这个目录存放了所有用户的秘密定时任务，所以它的权限极高，普通用户连用 `ls` 查看这个目录的权限都没有。只有 root 用户配合 `cat` 才能进去看。

```

SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root
HOME=/

# run-parts
01 * * * * root run-parts /etc/cron.hourly
02 4 * * * root run-parts /etc/cron.daily
22 4 * * 0 root run-parts /etc/cron.weekly
42 4 1 * * root run-parts /etc/cron.monthly

```

✨ 这道题考查的是通过输出结果的行文结构与特殊语法特征，来精准辨识 Linux 系统级定时任务配置文件。

本题的关键在于区分 用户级 crontab 和 系统级 `/etc/crontab` 的长相差异。

- `/etc/crontab`（系统级配置文件）：
 - a. 文件的开头会明确定义环境变量，如 `SHELL=`、`PATH=`、`MAILTO=`、`HOME=`。
 - b. 它的任务定义行一共有 7 个字段：前面 5 个是时间控制位，第 6 个字段是指定“以哪个用户身份运行”（如 `root`），第 7 个字段才是真正执行的命令（如 `run-parts ...`）。

开头带齐四环境变量，时间后面 root 站。七个字段少不了，此乃系统 crontab！

4. `at` 命令用于预约一次性任务，其核心管理选项如下：

- `-l`：显示（表示）当前处于预约队列中的所有任务。它和独立的 `atq` 命令完全等价。
- `-d` / `-r` 任务编号：删除（削除）指定的预约任务。它们和独立的 `atrm` 命令完全等价。

一次任务交给 `at`，管理选项靠字母：

`-l` 是 List 查列表（等同 `atq`），

`-d` (Delete) `-r` (Remove) 齐删掉（等同 `atrm`）。

查用 `q`，删用 `rm`，独立命令更省心！

📝 题目：23:00に1回限りのジョブを予約するコマンドは`at 23:00`

📝 考试小秘籍

a. 等价命令连线题：LPI 考试最喜欢出连线题或单选题考等价关系：

- `at -l` $\rightarrow$ `atq` * `at -d` / `at -r` $\rightarrow$ `atrm`
- 这两个对应关系在考场上就是送分题，必须秒杀。

- b. **时间格式**：正如图片下面小字提示的，`at` 后面接时间，最标准的格式是 `HH:MM`（如 `at 23:30`）。也可以用特殊的绝对时间，比如 `at now + 5 minutes`（5分钟后执行）。
- c. **与 cron 的核心区别**：大纲高频对比点：`cron` 负责周期性循环（每天都响的闹钟）；`at` 负责单次（响一次就扔掉的闹钟）。

查闹钟用小 `l` 或 `q`，取消闹钟用 `d`、`r` 或 `rm`。

5. crontabファイルの書式は

[分] [時] [日] [月] [曜日] [実行ユーザー名(システム用設定ファイルでのみ指定)] [コマンド]

項目	説明	表記	説明
分	0～59の範囲で指定	*	全ての値に一致
時	0～23の範囲で指定	-	範囲の指定。「時」に「1-3」と指定すると、1時から3時までを意味する
日	1～31の範囲で指定	,	複数の値の指定。「時」に「1,2,3」と指定すると、1時、2時、3時を意味する
月	1～12の範囲で指定	/	間隔の指定。「時」に「1-7/2」と指定すると、1時から7時の間で2時間おきを意味する
曜日	0～7の範囲で指定 (0と7=日曜、1=月、2=火、3=水、4=木、5=金、6=土)		

日時の指定（第1～第5フィールド）

在 `crontab` 的五个时间格子（分、时、日、月、星期）中，有两大考试最常考的隐藏规则：

- a. “每时间”的两种写法：在“小时”这一格，写 `*`（星号）和写 `0-23`（范围）在 Linux 系统眼里是完全等价的，都代表“从凌晨 0 点到晚上 23 点，每小时都执行”。
- b. “星期日”的双重身份：这是 Linux 定时任务里最奇葩的设定——数字 `0` 和数字 `7` 都代表星期日！系统两头都认。而数字 `6` 仅代表星期六。



例题：「hogescript」スクリプトを日曜の毎時間30分に実行したい。crontabファイルの適切な設定は次のうちどれか。（2つ選択）

30 * * * 0 hogescript

① 拆解第 1 个选项：30 * * * 0 hogescript

- 分：30 → 在第 30 分钟时
- 时：* → 每小时
- 日：* → 每天
- 月：* → 每月
- 星期：0 → 星期日
- 结果：完全符合题目要求的“在星期日的每小时 30 分执行”！所以它完全正确。

30 0-23 * * 7 hogescript

② 拆解第 6 个选项：30 0-23 * * 7 hogescript

- 分： 30 → 在第 30 分钟时
- 时： 0-23 → 0点到23点（这就是**每小时的**另一种高级写法）
- 日： * → 每天
- 月： * → 每月
- 星期： 7 → 星期日
- 结果：虽然写法不同，但翻译成大白话，和第 1 个选项的意思**一模一样**！所以它也完全正确。

6. 这道题是整个定时任务权限控制里**最经典的套路陷阱题**。

你纠结得完全有道理：“*test 既在白名单（allow），又在黑名单（deny），这不矛盾吗？系统到底听谁的？而且 root 怎么又冒出来了？*”

我们用一句话直接戳破这个纸老虎：**Linux 的权限判定非常死板，它根本不看冲突不冲突，它只按“谁的优先级高”这条死理走！**

在 `at` 和 `batch` 命令的访问控制中，系统的判定逻辑遵循以下**终极优先级**：

- 只要 `at.allow` 存在 🐼 系统直接无视 `at.deny`！只有名字写在 `at.allow` 里的普通用户才能使用。
- `root` 用户的特权 🐼 `root` 拥有至高无上的免死金牌。无论 `root` 有没有写进 `allow`，或者有没有被写进 `deny`，`root` 永远可以使用 `at`。

7. 为什么「`/etc/crontab`」にはコマンドの実行ユーザも指定する」是对的：因为

`/etc/crontab` 是**系统级**的文件，系统需要知道“到底让哪个用户去执行这条命令”，所以它必须包含**第 6 个字段——运行用户（実行ユーザ）**。

为什么「`/etc/crontab`」は**crontab**コマンドで設定する」是错的：`crontab` 命令（比如 `crontab -e`）是普通用户用来编辑**自己个人任务**的工具。而 `/etc/crontab` 是**核心系统文件**，必须由管理员直接用 `vi` 或 `nano` 等**文本编辑器**去直接修改，绝对不能用 `crontab` 命令来设定。

例：42 4 1 * * root run-parts /etc/cron.monthly

分 → 时 → 日 → 月 → 星期 → `root`（执行用户） → 命令

LPI 认证考试非常喜欢考这个“文字游戏”：

- 凡是看到 `crontab` 命令 → 它对应的一定是 `/var/spool/cron/` 目录。
- 凡是看到 `/etc/crontab` 文件 → 它对应的一定是 `vi` 等编辑器 并且 有 7 个字段（含用户名）。
- a. 这道题考查的是 `crontab` 配置文件中，系统是如何区分和识别“一条独立配置（1つの設定）”的。也就是问你：它是靠什么来当作每条任务之间的“分界线”的。

在 `crontab` 文件（无论是用户级还是系统级）中，每一行文本都代表一个独立的定时任务。

- **改行（换行符）**：Linux 系统严格按行读取配置文件。每遇到一个换行符（改行），系统就认为当前这条定时任务到此结束，下一行是一条全新的任务。
- **重要性**：如果没有换行符，所有的配置就会挤在同一行，系统会直接将其视为语法错误（因为超出了 6 个或 7 个字段的限制）。

为什么“改行”是必需的？

我们可以回顾一下之前看过的图片 `image_89338f.png` 的内容：

```
01 * * * * root run-parts /etc/cron.hourly
02 4 * * * root run-parts /etc/cron.daily
```

系统在读取这个文件时，就是通过每行末尾不可见的“换行符（改行）”，把这两条不同的任务完美隔开并分别独立识别的。所以，改行是识别“一个设定”的绝对标准。

crontabにおいて、1つの設定を識別するのに必要なものは改行

8. `anacron`は`cron`によるジョブスケジューリングを補完するツールです。`anacron`の特徴は以下の通りです。

- 日単位でジョブを実行する
- 実行履歴を管理しており、未実行のジョブを検出できる
- デーモン化（バックグラウンド実行）しないため、定期的に行う必要がある。主に`cron`によって実行される。
- ジョブ定義ファイルは `/etc/anacrontab`

这道题考查的是 `anacron`（异步定时任务管理系统）的核心特征、时间精度以及它与普通 `cron` 的关键区别。

💡 知识点总结

- `anacron` 的核心使命：它是专门为了经常关机、断电的电脑（如笔记本电脑）而设计的。如果电脑在某个定时任务本该执行的时间处于关机状态，`anacron` 会在下次开机（未起動のジョブを自動実行）时，自动把漏掉的任务补回来。
- **致命限制（时间精度）**：因为它的目的是“补课”，所以它的最小时间管理颗粒度是 **天（日单位）**。它无法做到像 `cron` 那样精确到几点几分（分单位）去执行任务。
- **工作原理（它不是常驻进程）**：`anacron` 自己并不会像 `cron` 那样作为一个守护进程（Daemon）长年累月地在后台挂着并定期检查。它是每次在**系统启动时**或者由 `cron` 顺带触发运行一次，检查完有没有漏掉的任务后就立刻退出。

为什么“ジョブ定義は `/etc/anacron/anacrontab` に記述する”是错的？

- 这是出题人非常阴险的**路径陷阱**。`anacron` 的全局配置文件确实叫 `anacrontab`，但它直接存放在 `/etc/` 目录下，也就是 `/etc/anacrontab`。中间并没有一个叫 `anacron/` 的子目录！
10. 以下は`cron`および`at`、`batch`のアクセス制御に使用されるファイルをまとめたものです。

cronの	許可	/etc/cron.allow
アクセス制御	拒否	/etc/cron.deny
at、batchの	許可	/etc/at.allow
アクセス制御	拒否	/etc/at.deny

11. crontab [オプション]

オプション	説明
-e (edit)	設定ファイル(crontabファイル)を編集
-l (list)	設定ファイルを表示
-r (remove)	設定ファイルを削除

12. batchコマンドは、システムの負荷の少ないタイミングで指定した時間に1回だけコマンドを自動実行します。

batchコマンドで設定されたジョブの管理は、atコマンドを使用していきます。

atコマンドと以下のオプションで、ジョブの表示や削除が可能です。

オプション	説明
-l	予約中のジョブを表示 (atqコマンドと同様)
-d ジョブ番号 -r ジョブ番号	指定した予約中のジョブを削除 (atrmコマンドと同様)

13. 这道题考查的是 `anacron` 配置文件 `/etc/anacrontab` 中，用来限制任务只能在特定时间段（时间窗）内执行的全局变量名和设置规范。

💡 知识点总结

- **避免高峰冲突**：由于 `anacron` 会在开机后补跑错过的定时任务，如果用户在白天高强度办公时开机，`anacron` 突然在后台疯狂跑备份脚本，就会导致电脑卡顿。
- **START_HOURS_RANGE**：这就是用来限制 `anacron` 只允许在每天的几点到几点之间被触发执行的全局变量。
- **时间窗逻辑**：它的赋值格式是 开始小时-结束小时。系统利用率最低的深夜通常是午夜到凌晨这段时间（比如 0-4 代表凌晨 0 点到凌晨 4 点之间）。
-

ローカライゼーションと国際化

1. 7ビットの文字コードで、128種類の文字を表現できるものはASCII

文字コード	説明
ASCII	7ビットの文字コード。 128種類の文字(英数字、記号、制御コード)を表現
ISO-8859	ASCIIを8ビットに拡張したもの。256種類の文字を表現
ISO-2022-JP	日本語でのメール送信に主に使用されていた日本語文字コード (JISコードとも呼ばれる)
EUC-JP	UNIX環境で標準的に利用されていた日本語文字コード
Unicode	多言語を扱うために作成された符号化文字集合(文字コード表) ※Unicodeで定義された文字を表現する符号化方式には、 UTF-8(多くのLinuxで標準的に利用される)、UTF-16、 UTF-32などがある

关键字符编码及其特征如下：

- **ASCII**：7位（7ビット）编码，表现128种文字（英数字、记号、控制码）。
- **ISO-8859**：将ASCII扩展为8位（8ビット），表现256种文字。
- **ISO-2022-JP**：主要用于日语邮件（メール送信），也叫 **JISコード**。
- **EUC-JP**：**UNIX环境**标准使用的日语文字编码。
- **Unicode**：为处理**多语言**而设计的字符集。其编码方式包括 **UTF-8**（许多Linux的标准）、UTF-16、UTF-32。

阿斯 (ASCII) 7 位 128 (数字)，ISO (8859) 加 1 位变 256 (数字)。邮寄邮件用 2022 (ISO-2022-JP)，UNIX 标配 EUC (EUC-JP)。多语言全球用 Unicode，UTF-8 跑在 Linux

位数陷阱：记住 **ASCII 是 7 位**（128种），**ISO-8859 是 8 位**（256种）。这是最容易考数值的地方。

用途匹配：题目提到「メール」选 ISO-2022-JP；提到「UNIX」选 EUC-JP；提到「Linux 標準」选 UTF-8 (Unicode)。

别名关联：看到「JISコード」要能反应出它是 ISO-2022-JP。

2. 这道题考查的是 Linux 中控制语言环境（Locale）的一系列环境变量及其优先级。

ロケール (Locale)：定义了特定语言、国家/地区的显示习惯，如字符编码（LC_CTYPE）、时间格式（LC_TIME）、货币符号（LC_MONETARY）等。

根据题目要求：“设定作为默认使用的值（デフォルトとして使用される値）”，且“允许按类别进行单独设置（カテゴリ毎に個別の設定は行える）”，分析如下：

- **LANG**：正是用于设定**全局默认值**的变量。当你设置了 `LANG=ja_JP.UTF-8` 后，所有的 `LC_*` 变量都会默认使用这个值，但你依然可以单独修改某个类别（如设置 `LC_TIME=en_US.UTF-8`）。

根据题目要求：“设定作为默认使用的值（デフォルトとして使用される値）”，且“允许按类别进行单独设置（カテゴリ毎に個別の設定は行える）”，分析如下：

- **LANG**：正是用于设定**全局默认值**的变量。当你设置了 `LANG=ja_JP.UTF-8` 后，所有的 `LC_*` 变量都会默认使用这个值，但你依然可以单独修改某个类别（如设置 `LC_TIME=en_US.UTF-8`）。
- **LC_NUMERIC / LC_CTYPE / LC_TIME**：这些变量属于**特定类别**的设置，只能影响它们各自的领域（数字格式、字符类型、时间格式），不能作为“默认值”去影响其他类别。
- **LC_ALL**：虽然它也能一次性设定所有类别，但它**不满足**题目的后半部分条件。因为 `LC_ALL` 的优先级太高，一旦设定，你再怎么单独设置 `LC_TIME` 也是无效的。

考试小秘籍

- 关键词匹配**：看到「デフォルト (Default)」找 `LANG`；看到「全てを上書き (覆盖全部)」找 `LC_ALL`。
- 编码确认**：在现代 Linux（如图片 `image_be47de.png` 所示）中，`LANG` 通常被设置为 `UTF-8` 编码以支持多语言。
- 常用命令**：想查看当前的ロケール设定，直接在终端输入 `localectl` 或 `locale` 即可。

環境変数(カテゴリ)	説明
LC_CTYPE	文字の分類、文字の種類の規定
LC_NUMERIC	数値の書式
LC_TIME	日付・時刻の書式
LC_MESSAGES	出力メッセージの言語
LC_MONETARY	通貨の書式
LC_ALL	指定した値が全てのカテゴリを上書きし、カテゴリ毎の設定ができない
LANG	指定した値が全てのカテゴリを上書きするが、カテゴリ毎の設定はできる (LC_ALLが設定されていない場合。指定した値がデフォルトとして使用される)

根据常见的 **ロケール用の主な環境変数**，核心类别及其含义如下：

- **LC_CTYPE**：文字の種類（Character Type）。决定字符分类、转换和编码（如 UTF-8）。
- **LC_MESSAGES**：メッセージ（Messages）。决定系统输出的提示信息、警告、菜单的语言。
- **LC_TIME**：時刻や日付（Time）。决定日期、星期、月份的显示格式。
- **LC_NUMERIC**：数字（Numeric）。决定数字的分隔符（如千分位点或逗号）。
- **LC_MONETARY**：通貨（Monetary）。决定货币符号（¥, \$）及金额显示格式。
- **LC_COLLATE**：照合順序（Collate）。决定字符串排序（如文件名的排序规则）。

CTYPE (C-Type) 编码定字符，MESSAGES (Message) 提示说话语。

TIME (Time) 看时间，MONETARY (Money) 钱，NUMERIC (Number) 数字小数点。

COLLATE (Collate) 排序排先后，默认 LANG，强制 ALL，记心间！

📖 考试小秘籍

- a. **字符集 vs 语言**: `LC_CTYPE` 决定的是你能否显示中文/日语（不乱码），而 `LC_MESSAGES` 决定的是系统对你说中文还是日语。
 - b. **排序陷阱**: 在 Linux 中，如果排序结果不符合预期（比如大写字母排在了小写字母后面），通常是因为 `LC_COLLATE` 的设置不同。
 - c. **最高指令**: 记得上一个考点吗？无论你单独怎么设这 6 类，只要设置了 `LC_ALL`，它们都会失效，全部听 `LC_ALL` 的。
3. 这道题考查的是“本地化（L10n）”与“国际化（I18n）”这两个易混淆概念的区别。

💡 知识点总结

- **L10n (Localization / 本地化)**: 将产品（软件、文档等）调整为适应特定国家或地区的语言、文化和习俗的过程。核心是“执行适配”。
- **I18n (Internationalization / 国际化)**: 在软件设计之初，就确保其能够灵活适配多种语言和地区，而无需对底层代码进行重大修改。核心是“架构设计”。
- **关系记忆**: I18n 是前提（把框架搭好），L10n 是结果（把翻译和格式填进去）。

答案：ソフトウェアで表示される言語やフォント、日付や通貨などの書式を利用者の言語に翻訳したりして合わせること

核心逻辑: 本地化（L10n）侧重于**具体的适配动作**。比如把英文菜单翻译成中文，把日期格式从 `MM/DD/YYYY` 改成 `YYYY/MM/DD`，把货币单位从 `$` 改成 `¥`。这完全符合第三项的描述。

选项4: ソフトウェア設計時から、多様な地域や多言語に対応できるように考慮すること

这是 **I18n（国际化）** 的定义。重点在于“设计时”和“考虑到多样性”。

想象你生产了一款**吹风机**:

- **I18n (国际化)**: 你在设计时，让吹风机可以支持 110V 到 220V 的电压，且电源线是可拆卸的。这就是“考虑了多样性”。
- **L10n (本地化)**: 当你要在日本卖这款吹风机时，你给它配了一个**日本标准的插头**，并印了一份**日文说明书**。这就是“具体的适配”。

口诀: 设计预留 I18n，具体翻译 L10n。

4. locale -a 命令

该命令会扫描系统的 locale 归档文件，列出所有可用的配置（如 `en_US.utf8`、`ja_JP.utf8` 等）。

字母含义: a 即 all，非常直观。

区分 locale 与 iconv: 看到 **locale** 想语言环境（L10n）；看到 **iconv** 想文件内容编码转换（如 UTF-8 转 EUC-JP）。

文字コードを変換できるコマンドは**iconv**

ロケール情報を表示するコマンドは**locale**

locale -a 查看系统中**所有可用**的语言环境

locale -m 查看系统支持的所有**字符编码 (charmaps)**

但是要注意区别的是：現在のロケールの設定が確認できるコマンドは**locale**

5. ロケールの設定において、全てのカテゴリをまとめて定義し、カテゴリ毎の個別の設定ができない環境変数は**LC_ALL**

根据题目要求：“将所有类别汇总定义（まとめて定義）”，且“无法按类别进行个别设置（個別の設定ができない）”，分析如下：

- **LC_ALL**：它是**强制性全选**。如果执行了 `export LC_ALL=ja_JP.UTF-8`，那么即使你再执行 `export LC_TIME=en_US.UTF-8`，系统的时间格式依然会显示为日语。这种“霸道”的特性使得它无法实现个别设置。
- **LANG**：虽然也汇总定义了默认值，但它**允许**被个别设置覆盖。

变量	优先级	是否允许子类覆盖？
LC_ALL	1 (最高)	否（强制统一）
LC_各类别	2 (中等)	不适用（自身即为特定设置）
LANG	3 (最低)	是（仅作参考值）

6. **LANG=C**：这是一个特殊的“标准”罗尔，旨在消除翻译，直接输出原始英文，常用于脚本解析。（「LANG=C」を設定すると、ロケールが異なる環境でも英語で共通の出力が得られる）

「/etc/sysconfig/i18n」：这是 **旧版本 (sysvinit 时代)** 的 CentOS/RedHat 系统的路径（如 CentOS 6 及以前）。

题目前提：题目明确提到了「**systemdが動作するシステム**」，因此必须选择符合 systemd 标准的路径，即 **/etc/locale.conf**。

输出一致性：无论你的系统原本是中文还是日语，一旦执行 `LANG=C`，所有的错误信息、时间格式都会变回最原始的英文。

脚本解析：在编写自动化脚本时，为了防止因为语言不同（比如“文件未找到”在日语里是“ファイルが見つかりません”）导致脚本失效，通常都会先设置 `LANG=C` 来获得**共通的英文输出**。

7. 例えばWindowsで作成したファイルをLinux上で開くと文字化けを起こすことがあります。このような場合は文字コードの変換を行います。文字コードを変換するには「iconv」コマンドを使用します。

iconvコマンドの書式と主なオプションは以下の通りです。

`iconv [オプション] [入力ファイル]`

オプション	説明
-l	扱える文字コードを表示 (list)
-f 文字コード	変換前の文字コードを指定 (from)
-t 文字コード	変換後の文字コードを指定 (to)

iconv 命令的核心选项如下：

- **-l: list**。列出（表示）系统目前能够处理的所有文字代码。
- **-f 文字コード: from**。指定转换前（源文件）的文字代码。
- **-t 文字コード: to**。指定转换后（目标）的文字代码。

iconv 转换文字码，选项全在单词里：

-l (List) 列出全名单，**-f (From)** 来自旧编码，**-t (To)** 转去新格式。

8.

anacron 的作用：与二十四小时开机的 `cron` 不同，`anacron` 专为“经常关机的电脑或笔记本”设计。它在开机后会去检查过去有哪些定时任务因为关机漏掉了，并自动补回来。

为什么要随机延迟：如果电脑一开机，所有漏掉的任务同时排山倒海般地涌上来，CPU 可能会瞬间飙到 100%。为了错开峰值，系统引入了全局随机延迟变量。

RANDOM_DELAY：这就是控制全局最大随机延迟时间的变量名。

单位规范：它的单位固定是 分（Minute），并且在赋值时只能写纯数字，绝对不能带 `min` 这种单位后缀。